A Case of Two Languages

Dr. Christian Rinderknecht

20 May 2026

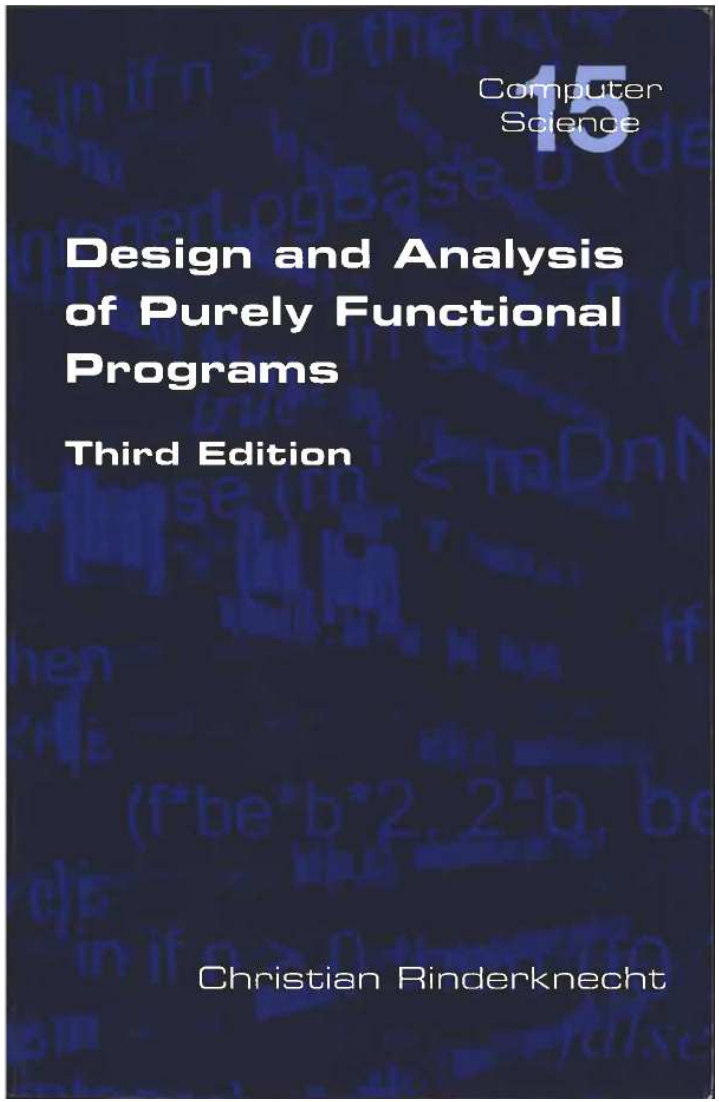
Eszterházy Károly Katolikus Egyetem
Eger (Hungary)

A personal introduction

- My alma mater is *Université Pierre et Marie Curie* (UPMC, a.k.a. Paris 6 in France).
- I did my doctoral studies (1993-1998) at INRIA, one of the most prestigious research institutes in informatics in France.
- I was a member of the team that has been developing the programming language OCaml. I am still in touch with them.
- I went on to work as an engineer, a researcher and a professor for many years, across several countries (France, Korea, Hungary, Sweden). I also worked in private companies.
- My main expertise domains are the design of Domain-Specific Languages (e.g., for abstract neuron networks or blockchains), compiler construction, functional programming and didactics of programming languages.

A personal introduction

My book about functional programming and the analysis of algorithms is published by King's College London (600 pages).
Revised in 2025.



Compiler front-ends – The case of ASN.1

My doctoral studies were dedicated to applying formal logic from the theory of programming languages to an industrial language in use in telecommunications.

The wide variety of hardware architectures and programming languages calls for the definition of a universal format for data to be exchanged over a computer network. *Abstract Syntax Notation One* (ASN.1) is a standardised language (ISO, ITU-T) for defining data types whose values may be exchanged across a network between two, possibly heterogeneous peers.

ASN.1 is completed by a set of *encodings* which serialise the values into streams of bits, the two most prominent being

- the Basic Encoding Rules (BER, and derived CER/DER),
- and Packed Encoding Rules (PER).

Application domains of ASN.1

- Telecommunication Management Network (SNMP, CMIP);
- Lightweight Directory Access Protocol (LDAP);
- email (X.400);
- UMTS (Radio Access Network);
- Transferred Account Procedure 3 (billing for GSM/UMTS roaming);
- 3GPP LTE (Long Term Evolution of GSM/UMTS networks);
- Multimedia conferencing systems with H.323 (for signalling: H.225, H.235, H.245, H.248, H.450.1);
- Interactive television services (MHEG);

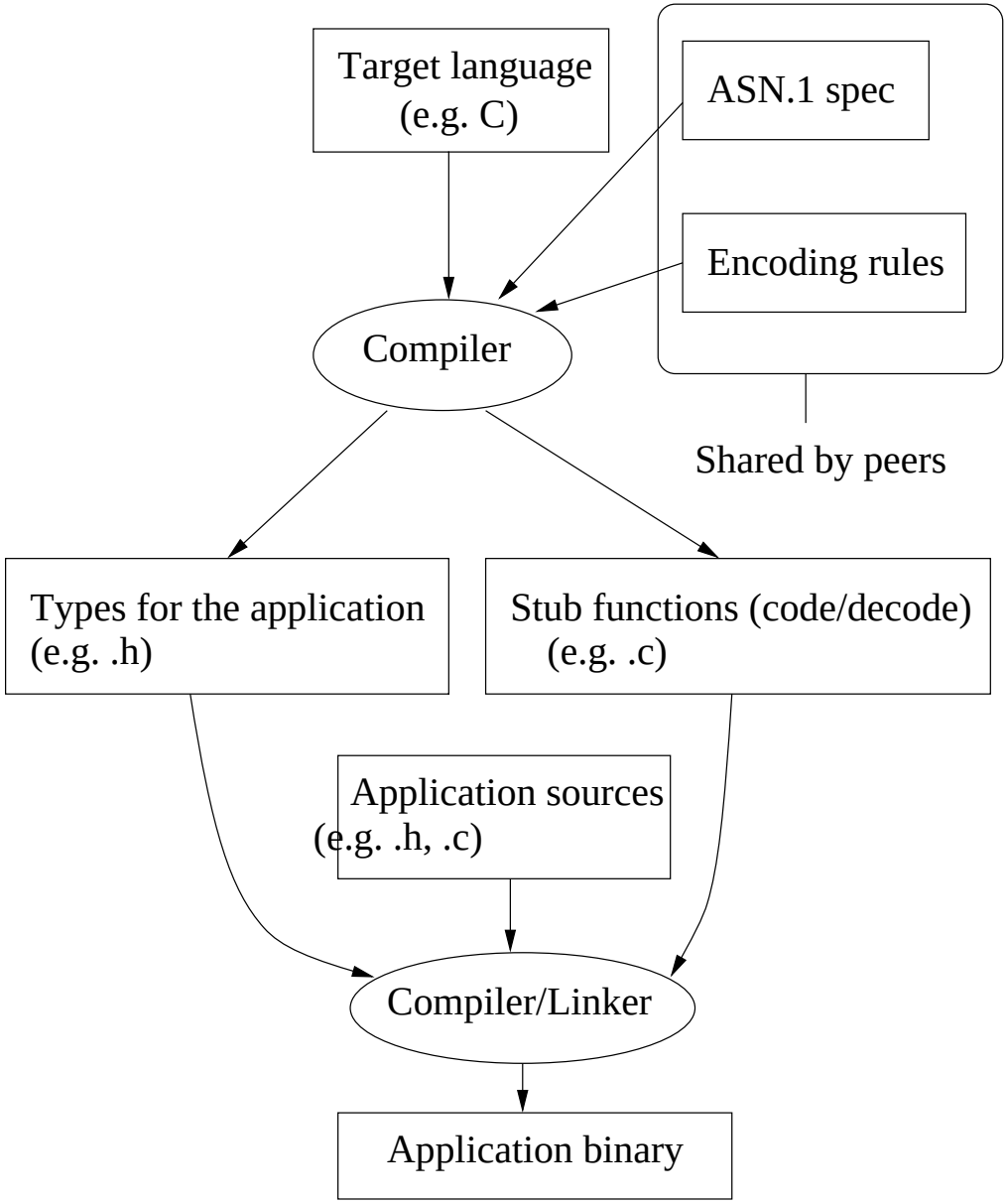
ASN.1 (usages, continued)

- Intelligent Transportation Systems (traffic signal control, electronic toll collection, emergency response etc.);
- Radio Frequency Identification (RFID);
- Aeronautical Telecommunication Network (air traffic control, flight information services, passenger communications, air-ground protocols etc.);
- Cryptography for electronic commerce over the internet (SET, PKCS 7, OpenSSL);
- Smart cards (SIM, WIM and VITALE);
- National Center for Biotechnology Information (NCBI): specification of millions of DNA sequences;

ASN.1 (usages, continued)

- client-server protocol for database access (U.S. Library of Congress: Z39.50);
- ASN.1 is embedded in other specification and programming languages like System Design Language (SDL, control), Message Sequence Charts (MSC, functional requirements), Testing and Test Control Notation (TTCN-3, tests suites), etc.
- ASN.1 is also used without formal control specification by protocol designers and tool implementors, *e.g.*, SNMP, OpenSSL.

ASN.1 and the compilation chain



How do ASN.1 and the BER interact?

- Both peers share a common ASN.1 specification and agree upon the usage of the BER;
- each peer compile the ASN.1 specification to a set of type declarations and a set of coders/decoders (codecs) for each type;
- the target language may be different depending on the peer;
- the coders transform a value into a stream of bits according to the BER,
- the decoders transform a stream of bits into a value according to the BER (or reject it as ill-formed);
- these pieces of source code are compiled and linked separately against the corresponding communicating applications.

General issues

1. Specifications: huge (700 pages A4, 10pt), complex (concepts mutually dependent).
2. Syntax: large and ambiguous grammar, dynamic extensions.
3. English cannot tell the meaning of all combinations of syntactical constructs.
4. Types denote value sets and type operators denote set operations: subtypes yield general set constraints.
5. Encoding only applies to a subset of ASN.1, defined implicitly and dependent on the encoding rules.
6. Decoding is unspecified.
7. Dynamic checks in codecs (depending on the target programming language) are unspecified.
8. Open source or free compilers: extremely rare, unmaintaned, fragile, without precise error reporting.

Syntax analysis

The BNF of ASN.1 is larger than that of ISO C++.

When fed to Yacc as it is specified in the standards, it yields between 3,000 and 4,000 shift/reduce and reduce/reduce conflicts (for each kind!). Unsurprisingly, it is ambiguous.

The situation is actually much worse: many *tokens* are ambiguous, so if the lexer works without any extra knowledge from the parser, there are even more conflicts.

I first solved the problem of the syntax analysis on the 1990 standard, which is smaller, albeit very nasty as well.

Theoretical Problems

1. the types may have only infinite values:
   `T ::= SET {a T};`
2. some value declarations may be ill-typed:
   `v REAL ::= "";`
3. especially, some value references may be ill-typed, as
   `a VisibleString ::= b`
   `b INTEGER ::= 0;`
4. the subtype constraints may be inconsistent:
   `T ::= REAL (SIZE(7));`
5. the subtypes may be empty, as
   `T ::= SET ((SIZE (1)) ^ (SIZE (2))) OF REAL;`
6. the subtypes may have no value set:
   `T ::= REAL (ALL EXCEPT T).`

Problems (cont.)

In other words, the issues to settle are respectively:

1. the finiteness problem;
2. the typechecking problem;
3. the type compatibility problem;
4. the constraint consistence problem;
5. the non-emptiness problem;
6. the solvability problem.

I showed that finiteness and solvability are enough to get a full validation of ASN.1.

I designed an algorithm to extract set constraints from ASN.1 specifications.

Analysis

- type compatibility $\subseteq$ typechecking;
- constraint consistence, non-emptiness $\subseteq$ solvability (the system is solved when we construct explicitly the values of each subtype);
- typechecking $\subseteq$ solvability:

  $$\texttt{y INTEGER (0..9) ::= 1} \rightarrow \begin{cases} \texttt{y A ::= 1} \\ \texttt{A ::= INTEGER (0..9)} \\ \texttt{B ::= A (y)} \end{cases}$$

  where `A` and `B` are fresh type references.

So, finiteness and solvability are enough to get a full validation of ASN.1.

Set Constraints

Sets constraints are inclusions between expressions interpreted over the domain of sets of trees which may be recursively defined.

The ASN.1 types, by having a tree structure (technically, they are rational trees), fit perfectly in the usual domain of set constraints.

A set constraint has the form $E_1 \subseteq E_2$, where E_1 and E_2 are *set expressions*. Examples are 0 (the empty set), 1 (the set of all terms), α (a set-valued variable), $c(E_1, E_2)$ (a constructor application), and the union, intersection or complement of set expressions. Given a set of constructors $\mathcal{C}$, where each $c \in \mathcal{C}$ has arity $a(c)$, set expressions are defined by the grammar:

$$E ::= 0 \mid 1 \mid c(E_1, \ldots, E_{a(c)}) \mid E_1 \cup E_2 \mid E_1 \cap E_2 \mid \neg E_1.$$

Common Application

In general, set constraints are used in the static analysis of programs. For us, solving these constraints means constructing the smallest supersets of the sets of the computed values (least fixpoint).

For example, consider a program with two uses of a variable v. Assume that from one use we determine that v must be a number (that is, $v \subseteq \mathsf{Int} \cup \mathsf{Float}$) and from the other use that v cannot be an integer (namely, $v \subseteq \neg\mathsf{Int}$). Combining these constraints, we can infer that v must be a floating point number (to wit, $v \subseteq (\mathsf{Int} \cup \mathsf{Float}) \cap \neg\mathsf{Int} = \mathsf{Float}$).

Application to ASN.1

In order to apply the set constraints to ASN.1 we define specific constructors that model all the subtyping constraints (like parsing to obtain an abstract syntax tree).

Then, the problem consists in extracting set constraints from each subtype, such as their solving provide a mapping from the subtype to its set of values: this is a kind of *denotational semantics* of ASN.1.

As far as validation is concerned, if some constraints are unsolvable or the set of solutions is empty, them the specification must be rejected.

Aiken and Wimmers (1993) proposed the first solving algorithm for the class of constraints we are using, *but there are practical limitations.*

So, what is wrong with the BER?

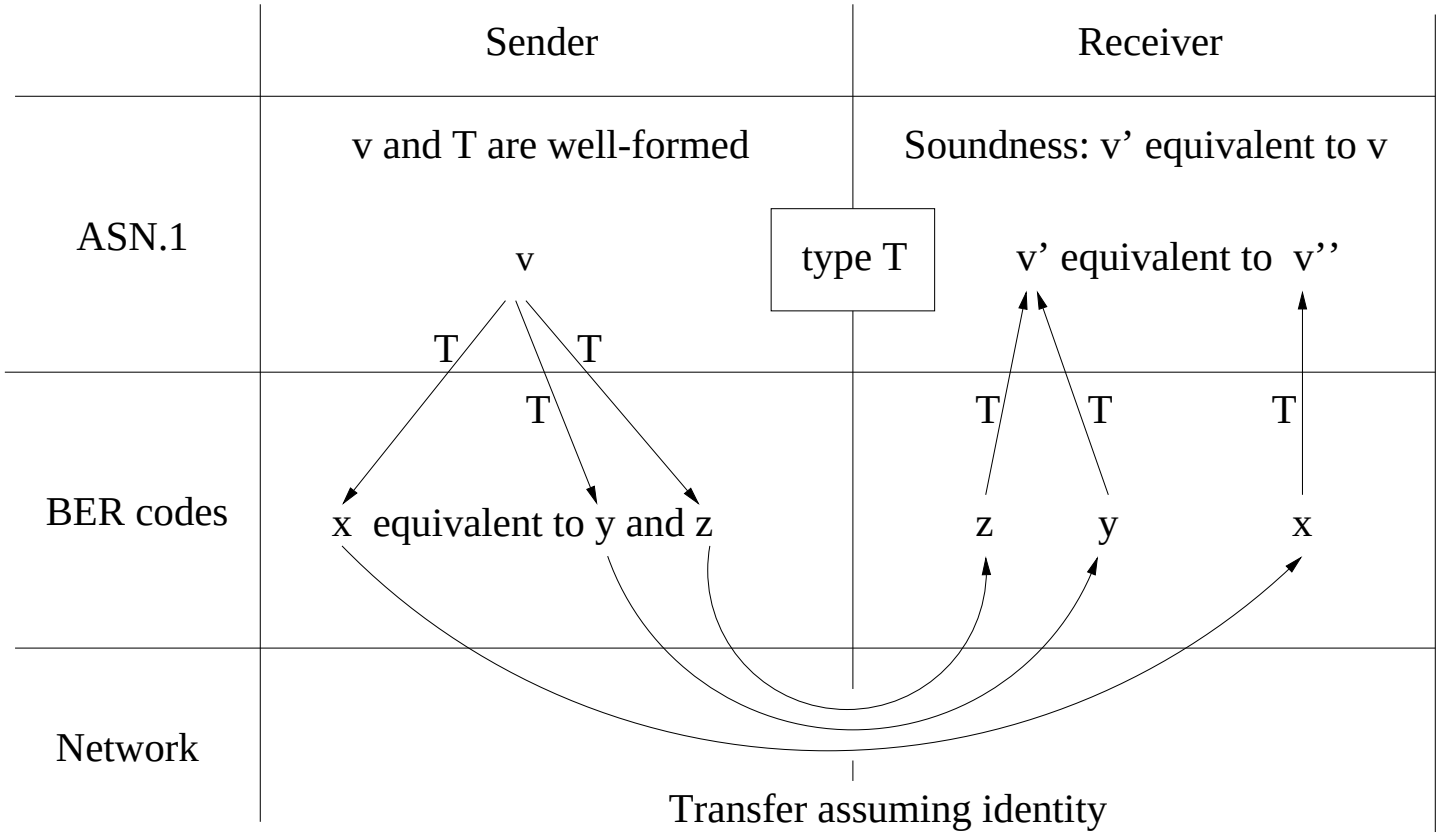
The BER are a set of rules defining the serialisation into bits of ASN.1 values.

Some years ago, the press reported several alleged vulnerabilities relating ASN.1 and the BER to SNMP and OpenSSL, about improper decoding (and rejection) of ill-formed BER encodings leading to

- buffer overflows,
- unspecified (non-deterministic) behaviours,
- call stack corruptions,
- hence possible denials of service.

I showed that the joint design of ASN.1 and the BER is sound, and that the problems lay with the back-ends of the compilers.

Soundness property



Compiler front-ends – The case of Tezos

- The economic protocol of Tezos handles a ledger of transactions as a **blockchain**. In a nutshell: a replicated database, immutable entries and resilience to malicious users.
- Those transactions consist in the exchange of assets (denoted on-chain by **tokens**) between peers of the network. Each peer has an address and an account (an amount of tokens), but **a physical person can be several peers.**
- **Public-key cryptography** is used to secure the identity of the senders of transactions and to ensure that past transactions are not tempered with. An address is the hash of the peer's public key.
- The economic protocol of Tezos enables the recording on-chain the business logic between peers, that is, the knowledge of what they agree should trigger transfers of tokens: **smart contracts**.

Smart Contracts on Tezos

- A smart contract is a peer that also has a program associated to a private storage.
- That storage is writable only by the program, but is publicly readable.
- The smart contract (by synecdoche) is executed each time a specific transaction is sent to its address. The transaction may include some parameters.
- Any call to a contract in a candidate block of transactions is replicated by all the nodes to check whether the block is valid before including it in their local view of the head of the chain.
- Once the block is validated, it is broadcasted by the underlying peer-to-peer, gossip network (as usual).
- The validation of smart contracts therefore adds a **delay** in the validation of a block: the execution time of all the smart contracts in it must fit the interblock time for the chain.

Smart Contracts on Tezos

- Therefore, each smart contract is allowed a given and fixed quantity of computation, measured in **gas**.
- Each instruction has an associated cost expected to be proportional to the wall-clock time and an *ad hoc* estimation is given by the node's client.
- A **node** is a server process that is accessed by a command-line client or RPCs. A node comprises a view of the chain so far, the **context of the chain** (a map from addresses to token amounts, used to validate transactions), and the **memory pool** (staging area).
- When the execution exceeds the alloted gas, it is stopped, its effect on the storage is rolled back, a failed transaction is included in the block, and thus fees are collected to **discourage spamming attacks**.
- The economic protocol sets limits on gas per block and per transaction.

Smart Contracts on Tezos

- Each smart contract has a **code size** and a **storage size**, which are allocated on every node on the chain (in their context).
- To limit the memory needed to synchronise the chain and store it, a fee is set per byte and collected when a contract is deployed (*originated*, in Tezos parlance).
- Smart contracts are **type-checked** before origination, which incurs a fee too.
- It is only at the normal termination of a contract that the **effects** of a contract become **atomically visible** to the network: the storage may appear modified and a list of operations may have been returned (transactions, contract creations and delegate settings) and validated.
- In particular, a smart contract can transfer tokens to other smart contracts, enabling the design of **complex distributed applications** in Tezos.

The design of Michelson

- The language native to the Tezos blockchain for writing smart contracts is **Michelson**, a domain-specific language inspired by functional and stack-based languages.
- This unusual lineage aims at satisfying unusual constraints, but entails some tensions in the design.
- First, to measure stepwise resource consumption, **Michelson is interpreted**.
- On the one hand, to assess resource usage per instruction, instructions should be simple, which points to **low-level features** (a RISC-like language).
- On the other hand, it was originally thought that users will want to write in Michelson by hand, so **high-level features** were deemed necessary (like a restricted variant of lambdas, a way to encode algebraic data types, and built-in sets, maps and lists).

The design of Michelson

- To avoid ambiguous and otherwise misleading contracts, the contents and layout of Michelson contracts has been constrained (e.g. identation, no UTF-8), and a **canonical form** was designed and enforced when storing contracts on the chain.
- To avoid the origination of contracts that would fail due to inconsistent assumptions on the storage and temporary values, a **strong static type system** was designed for Michelson.
- To reduce the size of the code, Michelson was designed as **a stack-based language**, whence the lineage from Forth and other concatenative languages like PostScript.
- Programs in those languages are **compact** because they assume an implicit stack in which some input values are popped, and output values are pushed, according to the current instruction being executed.

The semantics of Michelson

- The semantics of each Michelson instruction is determined by the change of a prefix of the stack, that is, a segment starting at the top.
- An abstraction of that change can be captured by the **type of the instruction**.
- For example, the instruction DUP has type $\alpha :: A \rightarrow \alpha :: \alpha :: A$, meaning that if the stack before the instruction is executed has a topmost item (also called *head*) of type α, where the substack (also called *tail*) has type A, then the instruction leaves a stack with two items of type α, on top of a stack of type A. (We use the OCaml notation for consing, that is, pushing items.)
- This does not specify entirely the semantics, only the **static semantics**.

The type system of Michelson

- We need a **dynamic semantics**, also called *evaluation* or *interpretation*, defined as a transition system, which states what happens to values in the stack. Here, the evaluation mimics exactly the typing judgement above:

$$\text{DUP } x :: S \;\rightarrow\; x :: x :: S$$

- Whilst the types of instructions can be polymorphic, their instantiations must be monomorphic, hence instructions are not first-class values and cannot be partially interpreted.
- This enables a simple **static type checking**, as opposed to a complex type inference. It can be performed efficiently: **contract type checking consumes gas**.
- Basically, type checking aims at validating the composition of instructions, therefore is key to safely compose contracts (concatenation) and refuse invalid data at run-time.

The type system of Michelson

- Once a contract passes type checking, it cannot fail due to inconsistent assumptions on the storage and other values (there are no null values, no casts), but it can still fail for other reasons: division by zero, token exhaustion, gas exhaustion, or an explicit FAILWITH instruction. This property is called **type safety**. Also, such a contract cannot remain stuck: this is the **progress property**.
- The existence of a formal type system for Michelson, of a formal specification of its dynamic semantics (evaluation), of a Michelson interpreter in the proof assistant Coq/Rocq, of proofs in Coq/Rocq of properties of some typical contracts, all those achievements are instances of **formal methods** in Tezos.

A voting contract in Michelson

- We want to write a smart contract in Michelson that records votes of peers for their favourite candidate.
- For the sake of simplificity, a candidate is denoted by a string.
- The vote is open to all peers for a fee, and they have to pick one candidate in a fixed list.
- The state of the stack at a given point will be written in a comment starting with #.

A voting contract in Michelson

- We begin by defining the storage as a mapping from the candidates (string) to the number of corresponding votes (int):

```
storage (map string int);
```

- Then we specify the type of the parameter of the contract, that is, the vote:

```
parameter string;
```

- The execution starts with a stack containing a single pair made of the parameter and the storage:

```
code {
# (vote, storage)
[...]
```

A voting contract in Michelson

- First, we check if the caller sent us enough tokens to vote, and, if not, we fail:

```
AMOUNT;
# amount::(vote, storage)
PUSH mutez 5000;
# 5000::amount::(vote, storage)
IFCMPGT
  {PUSH string "Insufficient fee."; FAILWITH} {};
# (vote, storage)
```

- The instruction `AMOUNT` pushes onto the stack the amount sent by the voter.
- The instruction `PUSH` pushes a typed constant.
- The instruction `IFCMPGT` compares the first two numbers on the stack, and, if the topmost is greater than the next, the first sequence (between braces) is executed, else the second one (which is empty here because of `FAILWITH`).

A voting contract in Michelson

- The instruction `FAILWITH` reads the value on top of the stack, terminates the execution, and signals an error (denoted by the value) to the node and the client.
- In order to check the current number of votes for the same candidate, we start by duplicating the topmost item:

```
DUP;
# (vote, storage)::(vote, storage)
```

- We then destructure the first pair:

```
UNPAIR;
# vote::storage::(vote, storage)
```

A voting contract in Michelson

- We can now consult the map in storage for the current count:

  ```
  GET;
  # (Some current | None)::(vote, storage)
  ```

  The top of the stack is `None` if `vote` was not a valid choice, and `(Some current)` if there were `current` votes already.

- If `None`, we must fail; otherwise, we increment the number of votes:

  ```
  IF_SOME { PUSH int 1; ADD; SOME }
     { PUSH string "Unknown candidate."; FAILWITH }
  # Some (current+1) :: (vote, storage)
  ```

- We need now to update the storage with the new vote. For that, we need to fish out the vote:

  ```
  DIP { UNPAIR };
  # Some (current+1) :: vote :: storage
  ```

  The instruction `DIP` applies its sequence of instructions (here `UNPAIR`) to the item just below the top.

A voting contract in Michelson

- Then we need to reorder the two first items, like so:

```
SWAP;
# vote :: Some (current+1) :: storage
```

- We can now update the map:

```
UPDATE;
# storage'
```

- All contracts must normally end by leaving in the stack a pair made of a list of operations and the new storage.
- In this instance, the list is empty because all we do is update the internal storage of the contract. So we push an empty list of operations and pair it with the new storage:

```
NIL operation;
PAIR;
# ([], storage')
}
```

The full Michelson contract

```
storage (map string int);
parameter string;
code {
 AMOUNT;
 PUSH mutez 5000;
 IFCMPGT { PUSH string "Insufficient fee."; FAILWITH } {};
 DUP;
 UNPAIR;
 GET;
 IF_SOME { PUSH int 1; ADD; SOME }
         { PUSH string "Unknown candidate."; FAILWITH }
 DIP { UNPAIR };
 SWAP;
 UPDATE;
 NIL operation;
 PAIR;
}
```

Assessment of the contract

- We could improve upon this design by enforcing a deadline on the ballot, or by granting the right to add new candidates in exchange of a fee.
- Anyhow, we had to modify, sometimes deeply, the stack in order to fetch and prepare the data for the instructions.
- Moreover, some instructions above were actually **macros** (`IF_SOME`, `IFCMPGT`), which are actually expanded into a sequence of atomic instructions by the Michelson parser, so the canonical contract on the chain is longer.
- Programming in **Michelson is fun** and might even save the Caps Lock key from extinction.
- *But does it scale up to larger and involved contracts with high stakes?*
- This is where **LIGO comes into play**.

LIGO

- Like Michelson, **LIGO is a programming language for writing smart contracts on Tezos.**
- Unlike Michelson, LIGO is a language akin to what a mainstream programmer would expect.
- This means that LIGO features variables, expressions, function calls, data types, pattern matching etc.
- Nevertheless LIGO remains a DSL for the Tezos blockchain, so not all the usual bells and whistles of a mainstream language are available.
- It is hosted here: `https://ligolang.org/`

LIGO

- Perhaps the most striking feature of LIGO is that it comes in **different concrete syntaxes**, and even **different programming paradigms**.
- In other words, LIGO is not defined by one syntax and one paradigm, like imperative versus functional.
- There is **JsLIGO**, which is inspired by TypeScript, an imperative language.
- There is **CameLIGO**, which is inspired by the pure subset of OCaml, hence is a functional language with few keywords, where values cannot be mutated.

The voting contract in CameLIGO

```
type storage = (string, nat) map
type return = operation list * storage

let vote (ballot, s : string * storage) : return =
  if   Tezos.get_amount () <= 5000mutez
  then failwith "Insufficient fee."
  else
    match Map.find_opt ballot s with
      None -> failwith "Unknown candidate."
    | Some current ->
        [], Map.update ballot (Some (current + 1n)) s
```